



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251198
Nama Lengkap	Moses Merohito
Minggu ke / Materi	12 / Tuples

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

MATERI 1 Tuple Immutable

Tuple hampir sama seperti list karena sama-sama bisa menyimpan banyak jenis data dan elemennya dapat diakses menggunakan indeks angka. Bedanya, tuple tidak bisa diubah setelah dibuat (immutable), sedangkan list masih bisa ditambah, dihapus, atau diubah isinya.

Selain itu, tuple juga bisa dibandingkan dengan tuple lain dan dapat digunakan sebagai key pada dictionary. Hal ini karena tuple termasuk objek yang hashable. Sederhananya, hashable berarti sebuah objek memiliki nilai identitas khusus (hash) yang tetap selama objek tersebut tidak berubah. Karena nilainya tetap, Python bisa menggunakan objek tersebut untuk mencari data dengan cepat di dictionary atau set.

Tipe data seperti angka dan string termasuk hashable karena nilainya tidak berubah. Sebaliknya, list dan dictionary tidak hashable karena isinya bisa berubah sewaktu-waktu. Kalau kita membuat class sendiri di Python, objek dari class tersebut biasanya belum hashable secara otomatis, kecuali kita menambahkan method `__hash__()` sendiri.

Sintaks Penulisan Tuples :

Sintaks penulisan *tuple* :

```
>>> t = 'a', 'b', 'c', 'd', 'e'
```

Sintaks lain penulisan *tuple* dapat menggunakan tanda kurung:

```
>>> t = ('a', 'b', 'c', 'd', 'e')
```

Tuple dengan satu element, ditambahkan koma (,) dibelakang penulisan *tuple*.

```
>>> t1 = ('a',)  
>>> type(t1)  
<type 'tuple'>
```

Jika tidak menambahkan tanda koma, akan dianggap sebagai *string*.

```
>>> t2 = ('a')  
>>> type(t2)  
<type 'str'>
```

Model sintaks lain dengan menggunakan fungsi **built-in** *tuple*. Ketika tidak diisi dengan argument apapun, secara langsung akan membentuk *tuple* kosong.

```
>>> t = tuple()  
>>> print(t)  
()
```

Jika argumennya berupa urutan (*string*, *list*, atau *tuple*), akan mengembalikan nilai *tuple* dengan elemen-elemen yang berurutan.

```
>>> t = tuple('dutaawacana')  
>>> print(t)  
('d', 'u', 't', 'a', 'a', 'w', 'a', 'c', 'a', 'n', 'a')
```

Tuple merupakan nama dari constructor, kita tidak bisa menggunakannya sebagai nama variabel. Sebagian besar operator yang bekerja pada list bekerja juga pada *tupple*. Tanda kurung kotak [] menandakan indeks element dari *tuple*.

```
>>> t = ('a', 'b', 'c', 'd', 'e')  
>>> print(t[0])  
'a'
```

Untuk menampilkan rentang nilai dari element *tuple* dapat menggunakan:

```
>>> print(t[1:3])  
( 'b', 'c' )
```

Tuple bersifat *immutable* artinya element pada *tuple* tidak dapat berubah. Jika anda berusaha mengubah *tuple*, maka akan didapatkan error.

```
>>> t[0] = 'A'  
TypeError: object doesn't support item assignment
```

Element pada *tuple* tidak dapat dirubah tetapi dapat diganti dengan elemen lain.

```
>>> t = ('A',) + t[1:]  
>>> print(t)  
( 'A', 'b', 'c', 'd', 'e' )
```

MATERI 2 Membandingkan Tuples

Operator perbandingan pada Python dapat diterapkan pada *tuple* maupun struktur data berurutan lainnya seperti list, dictionary, dan set. Cara kerjanya dimulai dengan membandingkan elemen pertama dari masing-masing data. Jika elemen pertama bernilai sama, maka Python akan melanjutkan perbandingan ke elemen berikutnya secara berurutan hingga ditemukan perbedaan yang menentukan hasil akhir.

Metode `sort()` di Python menggunakan prinsip yang sama dalam proses pengurutan. Pengurutan dimulai dari elemen pertama setiap data, lalu jika nilainya sama, Python akan membandingkan elemen berikutnya untuk menentukan posisi urutannya.

Pada kasus yang lebih kompleks, Python menerapkan teknik bernama DSU (Decorate, Sort, Undecorate), yang terdiri dari tiga tahap berikut:

1. **Decorate** – Setiap elemen digabungkan dengan satu atau beberapa key yang akan dijadikan dasar pengurutan, biasanya dalam bentuk tuple.
2. **Sort** – Kumpulan tuple tersebut kemudian diurutkan menggunakan fungsi `sort()` bawaan Python berdasarkan key yang sudah ditentukan.
3. **Undecorate** – Setelah pengurutan selesai, elemen asli dikembalikan dengan menghapus key tambahan yang sebelumnya digunakan.

```
1
2 kalimat = 'but soft what light in yonder window breaks'
3 dafkata = kalimat.split()
4 t = list()
5 for kata in dafkata:
6     t.append((len(kata), kata))
7
8 t.sort(reverse=True)
9
10 urutan = list()
11 for length, kata in t:
12     urutan.append(kata)
13
14 print(urutan)
15
16
```

Pada **looping pertama**, Python akan membuat daftar berupa tuple, di mana setiap tuple terdiri dari panjang kata dan kata itu sendiri. Proses pengurutan dilakukan berdasarkan elemen pertama dari tuple, yaitu panjang kata. Jika panjangnya sama, maka akan dibandingkan elemen kedua (kata itu sendiri). Penggunaan **`reverse=True`** berfungsi untuk membalik urutan hasil sortir, sehingga urutannya menjadi dari yang paling panjang ke yang paling pendek (descending).

Pada **looping kedua**, daftar kata disusun ulang dalam bentuk tuple dan diurutkan berdasarkan abjad secara terbalik, dengan mempertimbangkan panjang katanya. Dalam kasus tersebut, empat kata terakhir akan diurutkan menurut urutan alfabet secara menurun. Sebagai hasilnya, kata seperti "what" akan muncul sebelum "soft" dalam daftar.

['yonder', 'window', 'breaks', 'light', 'what', 'soft', 'but', 'in']

List ini menampilkan kata-kata yang telah diurutkan berdasarkan panjangnya dari yang terpanjang hingga yang terpendek.

MATERI 3 Penugasan Tuple

Salah satu kelebihan Python adalah kemampuannya menggunakan tuple pada sisi kiri proses assignment. Dengan fitur ini, beberapa variabel dapat diisi sekaligus hanya dalam satu baris kode.

Misalnya, ketika terdapat list atau tuple yang berisi beberapa data, setiap elemen dapat langsung dimasukkan ke variabel yang berbeda sesuai urutannya. Python akan membaca elemen-elemen tersebut satu per satu berdasarkan indeks.

Fitur ini juga memungkinkan pertukaran nilai dua variabel dengan sangat sederhana tanpa perlu variabel tambahan. Python menganggap bagian kiri dan kanan assignment sebagai tuple, lalu mencocokkan setiap nilai dengan variabel yang sesuai berdasarkan posisi.

Namun, jumlah elemen pada kedua sisi assignment harus sama agar tidak terjadi error. Kemampuan ini sangat membantu, contohnya ketika memisahkan username dan domain dari alamat email menggunakan fungsi `split()`.

```
>>> m = [ 'have', 'fun' ]
>>> x, y = m
>>> x
'have'
>>> y
'fun'
>>>
```

Python akan menterjemahkan sintaks *tuple* dalam langkah-langkah sebagai berikut:

```
>>> m = [ 'have', 'fun' ]
>>> x = m[0]
>>> y = m[1]
>>> x
'have'
>>> y
'fun'
>>>
```

Tuple yang digunakan pada contoh diatas menggunakan statement penugasan disebelah kiri dan tidak menggunakan tanda kurung. Berikut ini merupakan contoh yang sama dengan menggunakan tanda kurung (*parentheses*).

```
>>> m = [ 'have', 'fun' ]
>>> (x, y) = m
>>> x
'have'
>>> y
'fun'
>>>
```

Tuple memungkinkan pula untuk menukar nilai variabel dalam satu statement.

```
>>>> a, b = b, a
```

MATERI 4 Dictionaries and Tuple

Pada Python, dictionary memiliki metode bernama `items()` yang digunakan untuk menghasilkan kumpulan pasangan data dalam bentuk tuple. Setiap tuple berisi key dan value dari dictionary tersebut. Hasil dari `items()` biasanya tidak memiliki urutan tetap karena dictionary tidak selalu menyimpan data sesuai urutan input.

Namun, karena hasil tersebut berupa list yang elemennya adalah tuple, data dapat diurutkan dengan mudah. Tuple di Python mendukung operasi perbandingan sehingga list tuple dapat disusun menggunakan proses sorting. Dengan mengubah dictionary menjadi list tuple terlebih dahulu, isi dictionary dapat ditampilkan dalam keadaan terurut berdasarkan key yang dimilikinya.

```
1 #Membuat dictionary dengan beberapa pasangan key-value
2 d = {'a': 10, 'b': 1, 'c': 22}
3
4 #Menggunakan metode items() untuk mengambil daftar tuple dari dictionary
5 t = list(d.items()) # Konversi hasil items() menjadi list
6 print("List tuple dari dictionary (sebelum diurutkan):")
7 print(t) # Output: [('b', 1), ('a', 10), ('c', 22)] - tidak berurutan
8
9 #Mengurutkan list dari tuple berdasarkan key (default sort pada elemen pertama dari tuple)
10 t.sort()
11 print("\nList tuple setelah diurutkan berdasarkan key:")
12 print(t) # Output: [('a', 10), ('b', 1), ('c', 22)]
13
14 #Penjelasan tambahan:
15 #- Fungsi sort() akan mengurutkan berdasarkan elemen pertama dari tuple (yaitu key)
16 #- Karena key adalah string, maka urutannya secara alfabet (ascending)
17
18 #Jika ingin mengurutkan berdasarkan value, bisa gunakan parameter key:
19 t_by_value = sorted(d.items(), key=lambda item: item[1])
20 print("\nList tuple diurutkan berdasarkan value (nilai):")
21 print(t_by_value)
```

MATERI 5 Multipenugasan dengan Dictionaries

Python menyediakan fitur tuple assignment yang dapat digunakan saat melakukan iterasi pada dictionary. Ketika metode `items()` dipanggil, dictionary akan menghasilkan sekumpulan tuple yang berisi dalam dua variabel, misalnya `key` dan `val`, sehingga `key` dan `value` dapat diakses sekaligus dengan lebih praktis.

Selain itu, jika ingin mengurutkan dictionary berdasarkan `value`, kita dapat membuat list baru yang berisi tuple dengan format (`value`, `key`). Karena Python membandingkan tuple mulai dari elemen pertama, proses pengurutan akan otomatis dilakukan berdasarkan nilai `value` tersebut.

```
# Membuat dictionary dengan beberapa pasangan key-value
d = {'a': 10, 'b': 1, 'c': 22}

# Menggunakan metode items() untuk mengambil daftar tuple dari dictionary
t = list(d.items()) # Konversi hasil items() menjadi list
print("List tuple dari dictionary (sebelum diurutkan):")
print(t) # Output: [('b', 1), ('a', 10), ('c', 22)] - tidak berurutan

# Mengurutkan list dari tuple berdasarkan key (default sort pada elemen pertama dari tuple)
t.sort()
print("\nList tuple setelah diurutkan berdasarkan key:")
print(t) # Output: [('a', 10), ('b', 1), ('c', 22)]

# Penjelasan tambahan:
# - Fungsi sort() akan mengurutkan berdasarkan elemen pertama dari tuple (yaitu key)
# - Karena key adalah string, maka urutannya secara alfabet (ascending)

# Jika ingin mengurutkan berdasarkan value, bisa gunakan parameter key:
t_by_value = sorted(d.items(), key=lambda item: item[1])
print("\nList tuple diurutkan berdasarkan value (nilai):")
print(t_by_value) # Output: [('b', 1), ('a', 10), ('c', 22)]
```

MATERI 6 Kata Yang Sering Muncul

Program berikut dibuat untuk menghitung seberapa sering setiap kata muncul pada teks drama *Romeo and Juliet* (Act 2, Scene 2) yang terdapat di file `romeo-full.txt`. Program bekerja dengan membaca isi file, menghapus tanda baca, mengubah seluruh huruf menjadi huruf kecil, kemudian memisahkan setiap baris menjadi kumpulan kata.

Setiap kata yang ditemukan akan disimpan ke dalam dictionary, dengan kata sebagai `key` dan jumlah kemunculannya sebagai `value`. Setelah semua teks selesai diproses, isi dictionary diubah menjadi list of tuples dengan format (`jumlah`, `kata`), lalu diurutkan secara menurun berdasarkan jumlah kemunculan. Terakhir, program menampilkan 10 kata yang paling sering muncul beserta total kemunculannya.

```

import string # Mengimpor modul string untuk menghapus tanda baca

# Membuka file teks Romeo dan Juliet
fhand = open('romeo-full.txt')

counts = dict() # Dictionary kosong untuk menyimpan jumlah kata

# Membaca file baris per baris
for line in fhand:
    # Menghapus tanda baca dari setiap baris
    line = line.translate(str.maketrans('', '', string.punctuation))
    # Mengubah semua huruf menjadi huruf kecil
    line = line.lower()
    # Memecah baris menjadi list kata
    words = line.split()

# Mengisi dictionary dengan kata dan frekuensinya
for word in words:
    if word not in counts:
        counts[word] = 1 # Jika kata belum ada, masukkan dengan nilai 1
    else:
        counts[word] += 1 # Jika sudah ada, tambahkan 1

# Membuat list tuple (jumlah, kata) untuk proses sorting
lst = list()
for key, val in counts.items():
    lst.append((val, key)) # Format (jumlah, kata)

# Mengurutkan list berdasarkan jumlah (descending)
lst.sort(reverse=True)

# Menampilkan 10 kata teratas yang paling sering muncul
for val, key in lst[:10]: # Iterasi hanya 10 item pertama
    print(val, key)

```

MATERI 7 Tuple Sebagai Kunci Dictionaries

Tuple dalam Python bersifat hashable, sehingga bisa dijadikan sebagai key dalam dictionary. Hal ini berbeda dengan list yang tidak bisa digunakan sebagai key karena sifatnya *mutable*. Keunggulan ini memungkinkan kita menggabungkan beberapa nilai menjadi satu key tunggal, misalnya pasangan (nama belakang, nama depan) untuk memetakan ke nomor telepon. Saat melakukan iterasi, tuple tersebut dapat langsung dipecah ke dalam dua variabel terpisah sehingga memudahkan proses pencetakan atau manipulasi data.

```

# Mendefinisikan nilai-nilai untuk nama belakang, nama depan, dan nomor telepon
last = 'nendya' # Variabel last menyimpan string 'nendya' sebagai nama belakang
first = 'dida' # Variabel first menyimpan string 'dida' sebagai nama depan
number = '0881112266' # Variabel number menyimpan string nomor telepon

# Membuat dictionary kosong untuk direktori telepon
directory = dict() # Inisialisasi dictionary kosong bernama directory

# Menggunakan tuple (last, first) sebagai key dalam dictionary
directory[last, first] = number
# Key-nya adalah ('nendya', 'dida'), dan valuenya adalah nomor telepon

# Melakukan iterasi pada dictionary, key-nya adalah tuple (last, first)
for last, first in directory:
    # Menampilkan nama depan, nama belakang, dan nomor telepon
    print(first, last, directory[last, first])

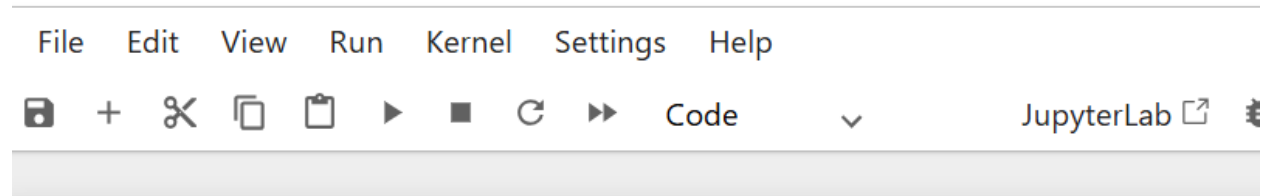
```

BAGIAN 2: LATIHAN MANDIRI (60%)

https://github.com/Moses808/71251198_Moses_Merohito

SOAL 1

 jupyter 71251198_week13 Last Checkpoint: 2 minutes ago



```
[1]: tA = (90, 90, 90, 90)

cek = True

for i in tA:
    if i != tA[0]:
        cek = False

print(cek)
```

True

Penjelasan :

- `tA = (90, 90, 90, 90)`
Membuat sebuah tuple bernama `tA` yang berisi beberapa angka.
- `cek = True`
Variabel `cek` digunakan untuk menyimpan hasil pengecekan.
Awalnya bernilai `True` karena diasumsikan semua anggota sama.
- `for i in tA:`
Melakukan perulangan untuk mengambil setiap nilai yang ada di dalam tuple.
- `if i != tA[0]:`
Membandingkan setiap anggota tuple dengan anggota pertama (`tA[0]`).
- `cek = False`
Jika ada satu saja nilai yang berbeda, maka hasil berubah menjadi `False`.

- `print(cek)`
Menampilkan hasil akhir:
 - `True` → semua anggota tuple sama.
 - `False` → ada anggota yang berbeda.

SOAL 2

jupyter 71251198_week13 Last Checkpoint: 12 minutes ago

```
File Edit View Run Kernel Settings Help
Python [conda env:base] *
[2]: data = ('MOSES MEROHITO', '71251198', 'PURWOKERTO, JAWA TENGAH')

print("NIM      :", data[1])
print("NAMA     :", data[0])
print("ALAMAT  :", data[2])

nim_tuple = tuple(data[1])
print("\nNIM      :", nim_tuple)

nama_depan = tuple(data[0].split()[0])
print("NAMA DEPAN:", nama_depan[:])

nama_terbalik = tuple(reversed(data[0].split()))
print("NAMA TERBALIK:", nama_terbalik)

NIM      : 71251198
NAMA     : MOSES MEROHITO
ALAMAT   : PURWOKERTO, JAWA TENGAH

NIM      : ('7', '1', '2', '5', '1', '1', '9', '8')
NAMA DEPAN: ('M', 'O', 'S', 'E', 'S')
NAMA TERBALIK: ('MEROHITO', 'MOSES')
```

Penjelasan:

- Pertama buat variabel yang berisi list data
- Setelah itu print indeks ke pertama (0) untuk menampilkan NIM
- Lakukan hal yang sama untuk menampilkan nama dan alamat sesuai indeks pada list
- Untuk menampilkan nim sesuai yang diminta, ubah list ke dalam tuple untuk memisahkan setiap karakter pada list yang diakses berdasarkan indeks. Lakukan hal yang sama untuk menampilkan nama yang dipisah per karakter. Namun bedanya adalah pada bagian nama, indeks paling kiri tidak di tampilkan maka menggunakan indeks dengan cara mengakses indeks mulai dari satu indeks dari paling kiri dengan cara [1:] hingga akhir. Gunakan fungsi tuple untuk memisahkan per karakter secara otomatis
- Untuk nama terbalik gunakan fungsi reverse untuk memutar karakter dari paling kiri menjadi paling kanan

- Setelah itu print hasil

SOAL 3

 jupyter 71251198_week13 Last Checkpoint: 32 minutes ago

```
File Edit View Run Kernel Settings Help
+ ✂ 📄 📄 ▶ ■ ↺ ⏪ Code ▾ JupyterLab 🌐
```

```
[2]: input_file = input("Enter a file name: ")

try:
    buka_file = open(input_file)
except:
    print("File tidak ditemukan.")
    quit()

hitung_jam = dict()

for line in buka_file:
    if line.startswith("From "):
        parts = line.split()
        waktu = parts[5]
        jam = waktu.split(":")[0]

        hitung_jam[jam] = hitung_jam.get(jam, 0) + 1

hasil = sorted(hitung_jam.items())
for jam, jumlah in hasil:
    print(jam, jumlah)
```

```
Enter a file name: mbox-short.txt
04 3
06 1
07 1
09 2
10 3
11 6
14 1
15 2
16 4
17 2
18 1
19 1
```

Penjelasan:

- Pertama buat try except untuk mengecek apakah file ada atau tidak, jika tidak maka print file tidak ditemukan namun jika ditemukan maka lanjut
- Setelah itu buat dictionary untuk menyimpan hasil looping
- Setelah itu looping setiap line yang ada di file setelah itu mulai cek setiap line yang berawal dari kata from.
- Setelah itu buat sebuah variabel untuk split line yang ada pada file
- Setelah itu akses indeks waktu dan jam di split dengan ":" dengan mengakses indeks 0
- Setelah itu panggil dictionaries dengan mengakses indeks jam yang sudah displit setelah itu ambil nilai jam. Jika nilai jam belum ada di dictionaries maka default nilainya adalah 0
- Setelah itu tambahkan satu ke dalam dictionaries untuk menambahkan dan menandai bahwa jam sudah muncul satu kali. Looping kondisi ini jika baris pada file sudah habis.

- Setelah itu buat variabel hasil untuk mengurutkan hasil looping nya
- Setelah itu print dictionaries dengan mengakses key dan value jam dan jumlah dalam variabel hasil Dimana hasil berisi dictionaries